

Ex. No: 14 File Allocation Strategies

PROGRAM A: SEQUENTIAL FILE ALLOCATION

C PROGRAM

```
#include <stdio.h>

int main()
{
    int start, length, i;
    printf("Enter Starting Block: ");
    scanf("%d", &start);
    printf("Enter File Length (Number of Blocks): ");
    scanf("%d", &length);
    printf("\nAllocated Blocks:\n");
    for(i = 0; i < length; i++)
    {
        printf("%d ", start + i);
    }
    printf("\n");
    return 0;
}
```

Output:

A terminal window on a Kali Linux system showing the execution of a C program for sequential file allocation. The user enters a starting block of 100 and a file length of 5. The program outputs the allocated blocks: 100, 101, 102, 103, and 104.

```
(kali@kali)-[~]
$ nano sequential.c
(kali@kali)-[~]
$ gcc sequential.c -o sequential
(kali@kali)-[~]
$ ./sequential
Enter Starting Block: 100
Enter File Length (Number of Blocks): 5
Allocated Blocks:
100 101 102 103 104
```

SHELL SCRIPT

```
#!/bin/bash
```

```

echo "Enter Starting Block:"

read start

echo "Enter File Length:"

read length

echo "Allocated Blocks:"

for ((i=0;i<length;i++))

do

echo -n "$((start+i)) "

done

echo

```

Output:



```

(kali㉿kali)-[~]
$ nano sequential.sh
(kali㉿kali)-[~]
$ chmod +x sequential.sh
(kali㉿kali)-[~]
$ ./sequential.sh
Enter Starting Block:
100
Enter File Length:
5
Allocated Blocks:
100 101 102 103 104

```

PROGRAM B: INDEXED FILE ALLOCATION

C PROGRAM

```

#include <stdio.h>

int main()

{

int n, indexBlock, blocks[20], i;

printf("Enter Index Block: ");

scanf("%d", &indexBlock);

printf("Enter Number of Blocks: ");

```

```

scanf("%d", &n);

printf("Enter Block Numbers:\n");

for(i = 0; i < n; i++)
{
scanf("%d", &blocks[i]);
}

printf("\nIndex Block : %d\n", indexBlock);

printf("Allocated Blocks : ");

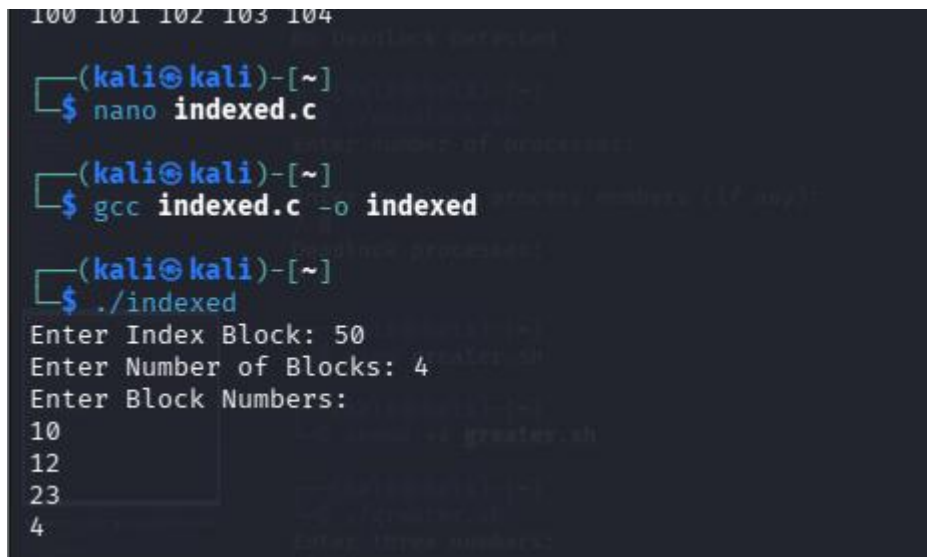
for(i = 0; i < n; i++)
{
printf("%d ", blocks[i]);
}

printf("\n");

return 0;
}

```

Output:



```

100 101 102 103 104
(kali@kali)-[~]
$ nano indexed.c
(kali@kali)-[~]
$ gcc indexed.c -o indexed
(kali@kali)-[~]
$ ./indexed
Enter Index Block: 50
Enter Number of Blocks: 4
Enter Block Numbers:
10
12
23
4

```

SHELL SCRIPT

```

#!/bin/bash

echo "Enter Index Block:"

read index

```

```

echo "Enter Number of Blocks:"

read n

echo "Enter Block Numbers:"

for ((i=0;i<n;i++))

do

read block[$i]

done

echo "Index Block : $index"

echo -n "Allocated Blocks : "

for ((i=0;i<n;i++))

do

echo -n "${block[$i]} "

done

echo

Output:

```

```

(kali㉿kali)-[~]
└─$ nano indexed.sh
Enter number of processes:
25
Enter block address numbers (if any):
37
48
Index block processed:
(kali㉿kali)-[~]
└─$ chmod +x indexed.sh
(kali㉿kali)-[~]
└─$ ./indexed.sh
Enter Index Block:
30
Enter Number of Blocks:
3
Enter Block Numbers:
13
45
67
Index Block : 30
Allocated Blocks : 13 45 67

```

PROGRAM C: LINKED FILE ALLOCATION

C PROGRAM

```

#include <stdio.h>

int main()

```

```

{
int n, blocks[20], i;

printf("Enter Number of Blocks: ");

scanf("%d", &n);

printf("Enter Block Numbers:\n");

for(i = 0; i < n; i++)

{

scanf("%d", &blocks[i]);

}

printf("\nLinked Allocation:\n");

for(i = 0; i < n - 1; i++)

{

printf("%d --> ", blocks[i]);

}

printf("%d --> NULL\n", blocks[n - 1]);

return 0;

}

```

Output:

```

(kali㉿kali)-[~]
└─$ nano linked.c
Enter number of processes:
(kali㉿kali)-[~]
└─$ gcc linked.c -o linked
(kali㉿kali)-[~]
└─$ ./linked
Enter Number of Blocks: 4
Enter Block Numbers:
100
200
300
400
Linked Allocation:
100 -> 200 -> 300 -> 400 -> NULL

```

SHELL SCRIPT

```
#!/bin/bash
```

```

echo "Enter Number of Blocks:"

read n

echo "Enter Block Numbers:"

for ((i=0;i<n;i++))

do

read block[$i]

done

echo "Linked Allocation:"

for ((i=0;i<n-1;i++))

do

echo -n "${block[$i]} --> "

done

echo "${block[$((n-1))]} --> NULL"

```

Output:

```

(kali㉿kali)-[~]
└─$ nano linked.sh
└─$ chmod +x linked.sh
└─$ ./linked.sh
Enter Number of Blocks: 3
Enter Block Numbers:
1
2
3
Linked Allocation:
1 → 2 → 3 → NULL

```